SIMATS ENGINEERING

ASSESSMENT 2 – SCENARIO-BASED QUESTIONS

COURSE CODE: CSA1509

COURSE NAME: CLOUD COMPUTING & BIG DATA ANALYTICS

Code of Conduct:

I, **Ashira Bridget Susan S (192524316),** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

CO2-Scenario-Based Questions

1. A university plans to virtualize its computer laboratory so that multiple operating systems can be provided on demand. Recommend a virtualization strategy, identify the suitable hypervisor type, and justify the decision.

2. A hospital cloud is facing VM escape and unauthorized snapshot access. Identify the major security risks and propose mitigation controls.

3. A retail company wants to modernize its datacenter using Software Defined Datacenter principles. Explain how compute, storage, and network virtualization would be introduced in phases.

4. Two cloud providers want to interoperate through federation. Explain how trust, identity, and privacy issues should be handled in the federation design.

5. An enterprise observes poor performance in a virtualized environment after overcommitting CPU and memory to multiple VMs. Analyze the scenario and recommend tuning actions.

CO2 Scenario-Based Questions: Virtualization, Security, SDDC and Federation

Cloud Computing Assessment | Course Outcome 2 (CO2)

Question 1: Virtualizing a University Computer Laboratory

Scenario summary

A university wants to serve multiple operating systems (Windows, Linux, possibly specialised OSes for networking, data science or programming courses) on demand from its computer lab, without maintaining many physical machines per OS.

Recommended virtualization strategy

Adopt **full (hardware-assisted) server virtualization combined with Virtual Desktop Infrastructure (VDI)**:

1. **Centralise compute.** Place a cluster of high-capacity servers in the datacenter, each running a hypervisor.
2. **Create golden images.** Build one master VM template per course OS/software stack (e.g., Windows with Visual Studio, Ubuntu with Python/ML libraries, a security-lab image).
3. **Provision on demand.** Use linked clones from templates so a student VM is ready in seconds, and delete or reset it after the session (non-persistent desktops). Keep persistent disks only for faculty or project work.
4. **Thin clients or browser access.** Lab PCs become thin clients (or students use their own devices through a remote display protocol such as RDP, PCoIP or HTML5 gateway), reducing hardware cost and maintenance.
5. **Isolation and resource control.** Use resource pools, per-course quotas, and VLAN segmentation so that one class's VMs (for example, a malware-analysis lab) cannot affect others.
6. **Scheduling and scaling.** Pre-power-on VMs before scheduled lab sessions and consolidate or shut them down afterward to save energy. Use shared storage plus live migration (vMotion-style) for load balancing and maintenance without downtime.

Suitable hypervisor type

A **Type 1 (bare-metal) hypervisor**, such as VMware ESXi, Microsoft Hyper-V, KVM (with a management layer such as oVirt/Proxmox) or Xen.

Justification

Criterion	Type 1 (bare-metal)	Type 2 (hosted)
Runs on	Directly on hardware	On top of a host OS
Performance	Near-native, low overhead	Extra overhead from host OS layer
Scalability	Dozens of VMs per host, clustering	Limited, suited to single desktops
Security	Smaller attack surface	Larger surface (host OS vulnerabilities)
Features	Live migration, HA, resource pools, centralised management	Mostly basic features
Typical use	Datacenters, VDI, production	Developer laptops, testing

A lab serving many concurrent students needs scale, performance, high availability and central management, which only a Type 1 hypervisor provides. Type 2 products (VirtualBox, VMware Workstation) are appropriate only for individual instructors prototyping images.

Additional considerations

- **Licensing:** KVM/Proxmox lowers cost for Linux-heavy labs; Hyper-V or ESXi suit Windows-centric environments.
- **GPU needs:** Courses such as AI or graphics may require GPU passthrough or vGPU.
- **Storage:** Use deduplication and thin provisioning because many VMs share the same base image.
- **Benefits:** Lower hardware and power cost, faster provisioning, easy rollback after student mistakes, simplified patching (update the template once), and remote access for students.

Question 2: Hospital Cloud Facing VM Escape and Unauthorized Snapshot Access

Scenario summary

A hospital cloud stores highly sensitive data (electronic health records, imaging, billing). Two threats are reported: **VM escape** (a guest breaking out of isolation to reach the hypervisor or other VMs) and **unauthorized access to VM snapshots** (which contain disk and sometimes memory state, including keys and patient data).

Major security risks

Risk	Description	Impact
VM escape	An attacker exploits a hypervisor, virtual device driver or shared-resource flaw to move from a guest VM to the host	Full compromise of the host and every co-resident VM; patient data exposure
Unauthorized snapshot access	Snapshots and backups copied, mounted or restored by unprivileged or malicious users	Data theft, offline credential and key extraction, tampering with clinical records
Hypervisor vulnerabilities	Unpatched hypervisor or management plane	Entry point for escape and takeover
Insecure management interfaces	Weak authentication on consoles and APIs	Attacker can create, clone, export or delete VMs and snapshots
VM sprawl and stale snapshots	Forgotten snapshots holding old, unpatched, sensitive data	Hidden data leakage and expanded attack surface
Side-channel attacks	Cache or timing attacks between co-located VMs	Leakage of cryptographic keys or data
Insider threat and weak access control	Over-privileged administrators	Data misuse
Regulatory exposure	Violations of HIPAA-style or national health-data regulations	Legal penalties and loss of trust

Mitigation controls

A. Preventing VM escape

1. Patch the hypervisor, firmware and virtual device drivers promptly through a controlled patch-management process.

2. Harden the hypervisor: disable unused services and virtual devices (floppy, serial, clipboard, drag-and-drop, shared folders), and use minimal configurations.
3. Use hardware-assisted isolation (Intel VT-x/AMD-V, IOMMU) and secure/measured boot with a TPM to verify hypervisor integrity.
4. Separate workloads by sensitivity: place high-risk or internet-facing VMs on different hosts or clusters from EHR databases (dedicated hosts or anti-affinity rules).
5. Deploy hypervisor-aware monitoring (introspection, IDS/IPS, anomaly detection) and centralised logging to a SIEM.
6. Reduce side-channel risk by avoiding co-tenancy for the most sensitive VMs and applying microcode mitigations.

B. Protecting snapshots and backups

1. **Encrypt** VM disks, snapshots and backups at rest (AES-256), with keys held in a hardware security module or key management service separate from the hypervisor.
2. **Role-based access control and least privilege:** only designated backup operators may create, export or restore snapshots. Enforce multi-factor authentication and approval workflows for exports and restores.
3. **Audit and alert:** log every snapshot creation, clone, mount and export; alert on unusual activity.
4. **Snapshot lifecycle policy:** limit snapshot age and count, auto-expire old ones, and securely delete them (crypto-shredding).
5. Store backups in **immutable, access-controlled storage**, with off-site copies.
6. Use encrypted VM memory features where available for memory snapshots.

C. Overall governance

- Secure the management plane: a dedicated management network, MFA, privileged access management, and no direct internet exposure.
- Network micro-segmentation to limit lateral movement if a VM is compromised.
- Regular vulnerability scanning, penetration tests and configuration baselines (such as CIS benchmarks).
- Staff training, an incident response plan, and compliance audits for health-data regulations.
- Defense in depth: no single control is relied upon.

Question 3: Modernizing a Retail Datacenter with Software Defined Datacenter (SDDC) Principles

Scenario summary

A retail company wants to replace rigid, hardware-centric infrastructure with a flexible, automated, policy-driven datacenter that can handle peak seasons (sales, festivals) and quickly roll out new applications such as e-commerce, point-of-sale and inventory systems.

SDDC concept

In an SDDC, all infrastructure (compute, storage, networking, and eventually security) is **virtualized, abstracted from hardware, and delivered as software-defined services**, managed through a unified automation and orchestration layer.

Phased adoption plan

Phase 0: Assessment and planning (preparation)

- Inventory applications, servers, storage and network; measure utilisation and dependencies.
- Define goals: cost reduction, agility, resilience during peak seasons.
- Choose a platform (VMware vSphere/vSAN/NSX, OpenStack with Ceph and Neutron, or a hybrid cloud service), plus monitoring and automation tools.
- Train staff and begin with a pilot.

Phase 1: Compute virtualization (foundation)

- Consolidate physical servers onto virtualized hosts using a Type 1 hypervisor.
- Migrate non-critical and test workloads first (physical-to-virtual conversion), then business-critical systems.
- Enable clustering, live migration, high availability and resource scheduling.
- *Benefits:* higher utilisation, lower hardware and power costs, quick VM provisioning, easier disaster recovery.

Phase 2: Storage virtualization (software-defined storage)

- Pool disks across servers (hyper-converged infrastructure such as vSAN or Ceph) or virtualize SAN/NAS arrays behind a single abstraction layer.
- Apply policy-based management (performance tier, replication, snapshot and backup policies), thin provisioning, deduplication and compression.
- Use tiering (SSD for hot data such as the transaction database, cheaper disks for archives and logs).

- *Benefits:* scalability on commodity hardware, no vendor lock-in to array hardware, automated data protection, simpler capacity growth.

Phase 3: Network virtualization (software-defined networking)

- Separate the control plane from the data plane; use overlay networks (VXLAN) and virtual switches, routers and load balancers.
- Introduce **micro-segmentation** and distributed firewalls, so that payment (PCI-DSS) systems are isolated from general workloads.
- Automate network provisioning with templates so new applications get networks and security policies automatically.
- Connect stores, branches and cloud through software-defined WAN.
- *Benefits:* faster network changes, stronger security, consistent policy across sites.

Phase 4: Automation, orchestration and self-service

- Add a cloud management platform with a service catalogue, infrastructure-as-code, and workflow automation so teams can self-provision.
- Integrate monitoring, capacity analytics, chargeback/showback, and policy compliance.
- Enable auto-scaling for seasonal peaks.

Phase 5: Hybrid extension and optimization

- Burst to a public cloud during peak sales events; use cloud-based disaster recovery.
- Continuously optimise cost, performance and security; consider containers and Kubernetes for modern applications.

Summary table

Phase	Layer	Key outcome
1	Compute	Server consolidation, HA, rapid VM provisioning
2	Storage	Pooled, policy-driven, scalable storage
3	Network	Programmable, segmented, secure networking
4	Automation	Self-service, orchestration, policy compliance
5	Hybrid	Elastic bursting and cloud DR

Risks and mitigations

Skill gaps (training), migration downtime (phased cutover with rollback plans), and integration complexity (pilot first and use open standards).

Question 4: Cloud Federation Between Two Providers

Scenario summary

Two cloud providers want to interoperate so that workloads, data and users can move or be shared across both clouds (for capacity, resilience or geographic reach). Because they are independent administrative domains, federation design must address **trust, identity and privacy**.

1. Handling trust

- **Establish formal trust agreements:** legal contracts and SLAs that define responsibilities, liability, audit rights and incident notification.
- **Technical trust anchors:** use a Public Key Infrastructure with certificates (mutual TLS) so each provider authenticates the other's endpoints and services; maintain a trusted certificate authority list and revocation checking.
- **Security baselines:** require each provider to meet agreed standards (ISO 27001, SOC 2) and undergo periodic audits and assessments.
- **Trust levels and policy mapping:** define which services, data classes and operations may cross the boundary. Translate security policies between providers so that equal protections apply.
- **Continuous monitoring and reputation:** shared logging, joint incident response, and the ability to suspend the federation link quickly if one party is compromised.
- **Open standards for interoperability:** use standard interfaces and formats (OCCI, CIMI, OVF, OpenStack/REST APIs) to avoid lock-in and ambiguity.

2. Handling identity

- **Federated identity management (FIM):** a user authenticates once with their home provider (identity provider, IdP) and the other provider (service provider, SP) accepts the assertion, enabling **single sign-on**.
- **Standards:** SAML 2.0, OpenID Connect and OAuth 2.0 for authentication and delegated authorization; SCIM for provisioning; Kerberos or X.509 where appropriate.
- **Attribute and role mapping:** map each provider's roles and attributes into a common model so that authorization is consistent (attribute-based and role-based access control).
- **Least privilege and short-lived tokens:** issue time-limited, scoped tokens; use multi-factor authentication for privileged and cross-cloud actions.
- **Lifecycle management:** synchronised de-provisioning, so revoking a user in the home domain removes access in the partner cloud promptly.
- **Auditability:** log identity assertions and cross-domain actions for non-repudiation.

3. Handling privacy

- **Data classification and minimisation:** share only the attributes and data needed; avoid exposing full identity attributes (use pseudonymous or anonymous identifiers, and selective attribute release).
- **Encryption:** encrypt data in transit (TLS/IPsec VPN) and at rest, with customer-controlled keys where possible.
- **Data residency and legal compliance:** respect jurisdictional rules (GDPR, national data protection laws); restrict placement and migration of regulated data to approved regions.
- **Consent and purpose limitation:** user consent for attribute release, clear data-processing agreements and defined retention and deletion policies.
- **Isolation:** keep tenant data separated within each provider and ensure the partner cannot access data beyond what is authorised.
- **Transparency:** privacy impact assessments, audit logs and breach-notification procedures covering both providers.

Summary

Issue	Key mechanisms
Trust	Contracts and SLAs, PKI/mutual TLS, audits, policy mapping, open standards
Identity	IdP/SP federation, SAML/OIDC/OAuth, role mapping, MFA, synchronised de-provisioning
Privacy	Data minimisation, encryption, residency controls, consent, audit trails

---

Question 5: Poor Performance After Overcommitting CPU and Memory

Scenario analysis

The enterprise assigned more virtual CPUs and memory to VMs than the physical hosts can actually supply. Overcommitment is normal in virtualization (it improves utilisation), but when too many VMs demand resources simultaneously, contention causes performance degradation.

Likely causes and symptoms

Resource	Problem	Typical symptom / metric
CPU	Too many vCPUs per physical core; VMs wait for scheduling	High **CPU ready time** (often above 5% is a warning, above 10% serious), high co-stop for multi-vCPU VMs, slow response, sluggish applications
Memory	Total allocated memory exceeds physical RAM	**Ballooning**, **host swapping**, memory compression, high page-fault rates, sharply increased latency
Storage	Swapping and contention increase disk I/O	High disk latency and queue depth
VM sizing	Oversized VMs (too many vCPUs) are harder to schedule	Poor performance despite low guest CPU usage
NUMA	VMs spanning NUMA nodes access remote memory	Higher memory latency
Noisy neighbours	A few heavy VMs starve others	Uneven performance across VMs

Memory overcommitment is usually more damaging than CPU overcommitment: once the hypervisor must swap guest memory to disk, performance can drop dramatically.

Recommended tuning actions

1. **Measure first.** Use monitoring tools (vCenter/esxtop, virt-manager/virsh, Prometheus, Zabbix or cloud monitoring) to identify CPU ready time, ballooning, swapping, and which VMs and hosts are saturated.
2. **Right-size VMs.** Reduce vCPUs and memory to what workloads actually use; start small and scale up. Avoid giving single-threaded applications many vCPUs.
3. **Lower overcommit ratios.** Keep CPU overcommit moderate (for example, roughly 3:1 to 4:1 vCPU to physical core for general workloads, lower for latency-sensitive ones) and avoid overcommitting memory for critical production VMs.

4. **Use reservations, limits and shares.** Reserve CPU and memory for critical VMs (databases, ERP); use shares to prioritise important workloads under contention; avoid unnecessary hard limits that cause artificial throttling.
5. **Balance the load.** Use DRS or equivalent schedulers and live migration to spread VMs across hosts; apply affinity/anti-affinity rules; separate heavy VMs.
6. **Scale capacity.** Add hosts or RAM to the cluster, or offload workloads to a public cloud, if demand genuinely exceeds capacity.
7. **Optimise memory.** Install guest tools/drivers (balloon driver) so reclamation works efficiently; enable transparent page sharing or compression where appropriate; avoid host swap by keeping adequate free memory; use large pages for memory-intensive applications.
8. **Respect NUMA.** Size VMs to fit within a NUMA node where possible and enable NUMA-aware scheduling.
9. **Tune storage.** Place swap and heavy-I/O VMs on fast (SSD/NVMe) storage and avoid I/O bottlenecks; check for snapshot buildup that slows disks.
10. **Schedule and govern.** Stagger batch jobs, backups and patching; define capacity policies and alerts; clean up idle or zombie VMs; review capacity regularly with trend analysis.

Conclusion

Overcommitment should be **managed, not eliminated**: monitor contention metrics, right-size workloads, protect critical VMs with reservations, balance load across the cluster, and add capacity when sustained demand exceeds supply.
